BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

BAKU HIGHER OIL SCHOOL

Department of Information Technology

Online Marketplace System

Object-Oriented Programming — Project Report

Course: Object-Oriented Programming

Faculty: Computer Science

Group: 5

Student 1: Mahammad Karimov

Student 2: Zakir Ibrahimov

Student 3: Eltun Asgarov

Instructor: Huseynov Shamistan

Submission Date: April 10, 2026

Contents

1 Abstract **3**

2 Introduction **4**

2.1 Problem Statement 4

2.2 Why This System Is Needed 4

2.3 Project Goals 4

2.4 Overview of the Solution 5

2.5 Role of OOP 5

3 Tools and Technologies Used **6**

3.1 Java 17 6

3.2 Spring Boot 3.2 6

3.3 Database and Persistence 6

3.4 Build Tool — Maven 7

3.5 Web Interface 7

3.6 Security 7

3.7 Testing 7

4 System Design and Architecture **8**

4.1 Architecture Overview 8

4.2 Four-Layer Architecture 8

4.2.1 Presentation Layer 8

4.2.2 Business Logic Layer 9

4.2.3 Model (Domain) Layer 9

4.2.4 Data Access Layer 9

4.3 System Flowchart — Order Checkout 10

4.4 Typical Operation Flow 10

4.5 User Interaction 11

5 Object-Oriented Concepts Used **12**

5.1 Classes and Objects 12

5.2 Encapsulation 13

5.3 Inheritance 14

5.4 Polymorphism . . . 15
5.5 Abstraction . . . 16
5.6 Interfaces . . . 17
5.7 Method Overloading and Overriding . . . 18
5.8 Exception Handling . . . 19
5.9 Static Members . . . 20
5.10 Collections . . . 21
5.11 Composition and Aggregation . . . 22

6 Program Workflow **23**
6.1 Application Startup . . . 23
6.2 Authentication . . . 24
6.3 Main Application Loop . . . 24
6.4 Operation Flow: Add to Cart . . . 26

7 Input and Output Examples **27**
7.1 User Registration . . . 27
7.2 Merchant Adds a Product . . . 27
7.3 Shopper Buys a Product . . . 28
7.4 Admin Views Platform Metrics . . . 29

8 Conclusion **30**
8.1 What Was Built . . . 30
8.2 OOP Usage . . . 30
8.3 System Strengths . . . 31

9 Future Improvements **32**
9.1 Real Payment Gateway Integration . . . 32
9.2 Product Search and Filtering . . . 33
9.3 Product Review and Rating System . . . 33
9.4 Merchant Analytics Dashboard . . . 34
9.5 Email Notifications . . . 34
9.6 REST API and Mobile Support . . . 35

10 References **36**

Chapter 1

Abstract

This report presents the design and implementation of a web-based Online Marketplace System developed as part of an Object-Oriented Programming (OOP) course project. The system allows three distinct types of users — shoppers, merchants, and administrators — to interact with a shared platform where products are listed, discovered, purchased, and managed. The project demonstrates how real-world e-commerce workflows can be modelled and implemented using core OOP principles.

Technologies Used: The backend is built entirely in **Java 17** using the **Spring Boot 3.2** framework. Data is persisted with **Spring Data JPA** over either an **SQLite** (development) or **PostgreSQL** (production) database, managed with **Flyway** migrations. The web layer uses **Thymeleaf** server-side templates with **Spring Security** handling authentication and role-based access control. Build automation is handled by **Maven**.

OOP Principles Applied: The codebase is structured around a set of well-defined entity classes in the `domain` package (representing real-world concepts like `User`, `Product`, `CustomerOrder`, `Cart`, and `Payment`). Encapsulation is enforced through private fields with accessor methods. Abstraction is achieved via service interfaces such as `PaymentService`, with concrete implementations like `MockPaymentService`. The `User` class implements the Spring Security `UserDetails` interface, demonstrating interface-based polymorphism. Exception handling is centralised through a `GlobalExceptionHandler`.

Final Outcome: The project delivers a functional multi-role marketplace with product listing approval, shopping cart management, order checkout, mock payment processing, dispute resolution, and an admin dashboard showing platform metrics. It serves as a clear practical demonstration of how OOP concepts translate into a real, deployable software system.

Chapter 2

Introduction

2.1 Problem Statement

Online commerce has become the dominant method of buying and selling goods worldwide. Despite this, many small businesses and individual sellers still lack access to an affordable, easy-to-use platform where they can list their products and reach customers. Equally, buyers need a trustworthy environment where they can browse products, manage a cart, and pay securely.

This project was motivated by the need to understand how such a system works from the ground up — not just as a user, but as a developer who must model real business logic in code.

2.2 Why This System Is Needed

A marketplace system is one of the most comprehensive applications a student can build because it requires integrating many software engineering concepts at once: data modelling, authentication, business rules, data validation, and user interface design. Understanding how to structure such a system cleanly — separating concerns, reusing logic, and preventing errors — is exactly what OOP is designed to help with.

2.3 Project Goals

The main goals of this project were:

- To implement a fully functional web-based marketplace system in Java.
- To apply core OOP principles — encapsulation, inheritance, polymorphism, abstraction — in a realistic context.
- To manage multiple user roles (shopper, merchant, admin) with different permissions.
- To handle the full lifecycle of a purchase: browsing, carting, checkout, payment, and fulfilment.

- To include administrative functions such as merchant approval, product moderation, and dispute resolution.

2.4 Overview of the Solution

The system was built as a Spring Boot web application. Users register either as shoppers or as merchants (who must be approved by an admin before they can sell). Once registered, merchants can create product listings with images. Shoppers can browse the catalogue, add products to a cart, and check out. The checkout process creates an order, and a mock payment service simulates a real payment gateway. Admins have a dashboard with platform metrics, the ability to approve or reject merchants, moderate listings, manage orders, and resolve disputes.

2.5 Role of OOP

OOP is the architectural backbone of the entire system. Every real-world entity — a user, a product, an order — is modelled as a class with its own data and behaviour. Services operate on these classes through well-defined methods. Interfaces define contracts (like the `PaymentService` interface), which allows the payment implementation to be swapped without affecting the rest of the system. This design makes the codebase modular, readable, and maintainable.

Chapter 3

Tools and Technologies Used

3.1 Java 17

The entire project is written in **Java 17**, a Long-Term Support (LTS) version of the language. Java was chosen for its strong type system, mature ecosystem, and excellent support for OOP. Features from modern Java such as enhanced switch expressions, text blocks, and record patterns improve code readability.

3.2 Spring Boot 3.2

Spring Boot provides a production-ready framework built on top of the Spring Framework. It removes boilerplate configuration and provides auto-configuration for web servers, security, and databases. The project uses the following Spring Boot starters:

- **spring-boot-starter-web** — Embeds a Tomcat server and provides MVC.
- **spring-boot-starter-data-jpa** — Object-relational mapping via Hibernate.
- **spring-boot-starter-security** — Authentication and authorisation.
- **spring-boot-starter-thymeleaf** — Server-side HTML templating.
- **spring-boot-starter-validation** — Bean validation for form inputs.

3.3 Database and Persistence

Data persistence is handled by **Spring Data JPA** (backed by Hibernate). Two database profiles are supported:

- **SQLite** — Used during local development. Lightweight, zero-configuration.
- **PostgreSQL** — Intended for deployment in a production or staging environment.

Database schema migrations are managed by **Flyway**, which applies versioned SQL scripts automatically at startup. This ensures the schema is always consistent with the

application version.

3.4 Build Tool — Maven

The project uses **Apache Maven** with a `pom.xml` descriptor. The Maven Wrapper (`mvnw`) is included so the project can be built without requiring a system-wide Maven installation.

3.5 Web Interface

The user interface is rendered server-side using **Thymeleaf** HTML templates. These templates are stored in `src/main/resources/templates/` and are organised by role (`admin/`, `merchant/`, `orders/`, etc.). A small amount of custom CSS is included in `static/css/style.css`.

3.6 Security

Spring Security is configured in `SecurityConfig.java`. It provides:

- Form-based login at `/login`.
- Role-based route protection — `/admin/**` requires `ROLE_ADMIN`, `/merchant/**` requires `ROLE_MERCHANT`, etc.
- **BCrypt** password hashing for all stored passwords.
- Method-level security enabled via `@EnableMethodSecurity`.
- CSRF protection enabled by default (with a specific exception for the H2 console).

3.7 Testing

Unit and integration tests are written with **JUnit 5** and **Spring Security Test**. Notable test classes include `OrderServiceStockTest` (verifies stock deduction logic) and `AdminControllerMvcTest` (tests admin endpoints using `MockMvc`).

Chapter 4

System Design and Architecture

4.1 Architecture Overview

The application follows a classic layered architecture, which is the most common and well-understood pattern for web applications. Each layer has a single clear responsibility, and communication flows downward — controllers call services, services call repositories, and repositories talk to the database.

4.2 Four-Layer Architecture

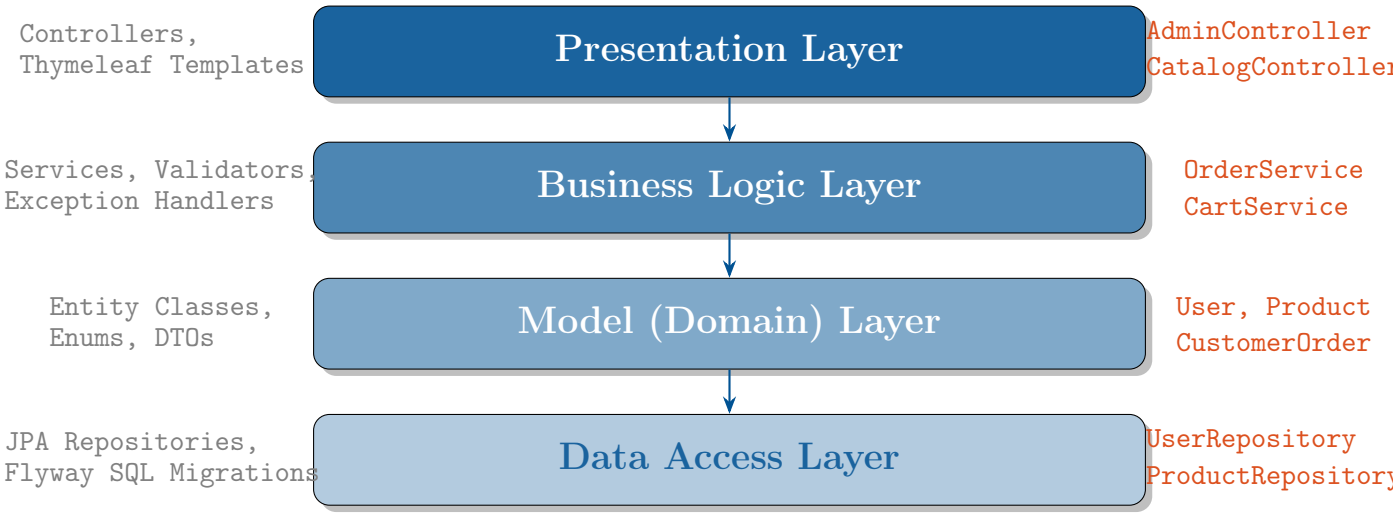

Figure 4.1: Four-Layer Architecture of the Marketplace System

4.2.1 Presentation Layer

This layer contains all Spring MVC `@Controller` classes and the Thymeleaf HTML templates. Controllers handle HTTP requests, delegate to services, and return view names or redirect responses. No business logic lives here.

4.2.2 Business Logic Layer

Service classes (`@Service`) contain all business rules: checking that a merchant is approved before they can list a product, verifying stock before checkout, applying role checks, and raising `BusinessException` when a rule is violated. This is where the application's intelligence resides.

4.2.3 Model (Domain) Layer

The domain package holds all JPA entity classes and enums. These are plain Java objects annotated with `@Entity`. They define the data structures and relationships between concepts: a `CustomerOrder` has many `OrderLine`s, a `Cart` belongs to one `User`, and so on.

4.2.4 Data Access Layer

Spring Data JPA repository interfaces (`ProductRepository`, `UserRepository`, etc.) provide database operations without requiring any SQL to be written manually. Flyway SQL migration scripts handle schema creation and updates.

4.3 System Flowchart — Order Checkout

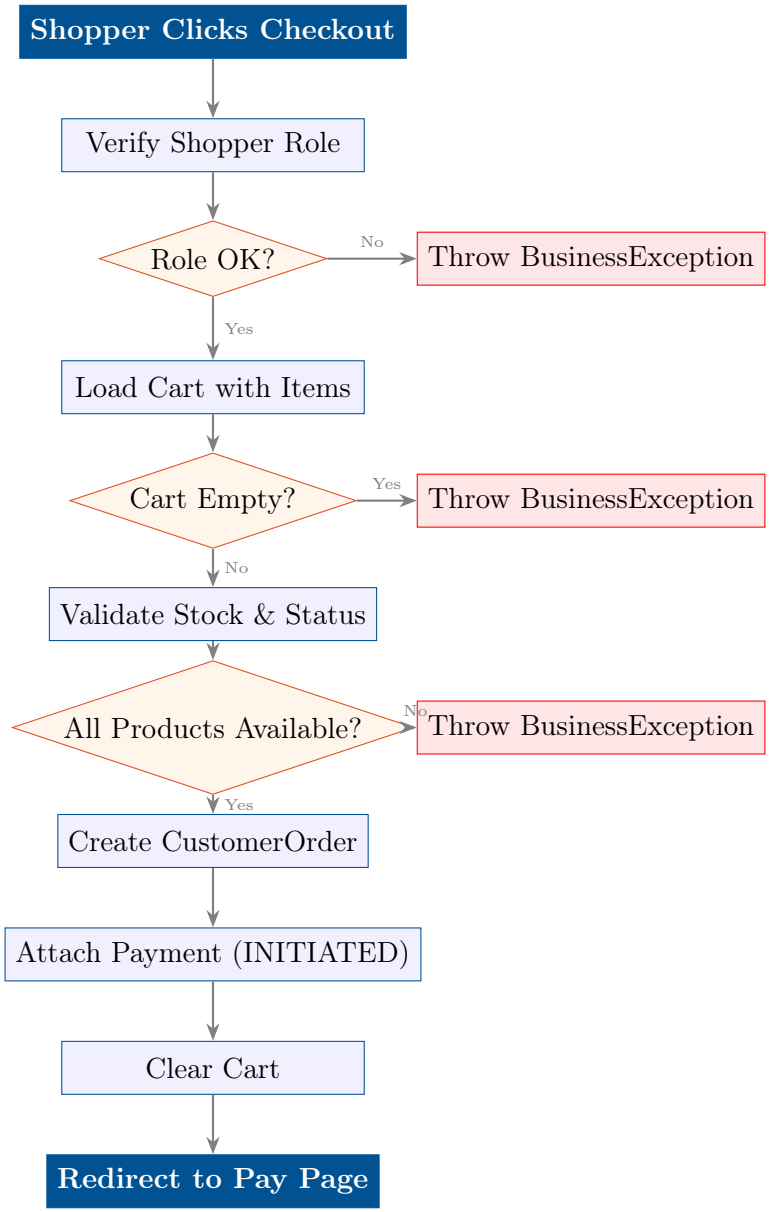

Figure 4.2: Checkout Process Flowchart

4.4 Typical Operation Flow

A typical session in the marketplace follows these steps:

1. A new user visits the site and registers as a *shopper* or as a *merchant*.
2. If registering as a merchant, an admin must approve the application before the merchant can list products.
3. An approved merchant logs in and creates product listings with titles, descriptions,

prices, stock counts, and up to five images.

4. New listings start in `PENDING_APPROVAL` status; an admin must change the status to `ACTIVE` before shoppers can see the product.
5. A shopper browses the catalogue, views product details, and adds items to their cart.
6. The shopper proceeds to checkout. The system validates stock and product availability, then creates a `CustomerOrder` with `PENDING_PAYMENT` status.
7. The shopper pays using the mock payment service. The system deducts stock and sets the order to `PAID`.
8. If a dispute arises, the shopper can raise it, and an admin resolves it.

4.5 User Interaction

Three distinct roles interact with the system through separate areas:

Role	Capabilities
Shopper	Register, browse catalogue, search products, manage cart, checkout, view orders, raise disputes.
Merchant	Register (with approval), create/edit/delete product listings, view own orders and analytics.
Admin	Approve/reject merchants, moderate product listings, view all orders, manage users, resolve disputes, view platform metrics.

Table 4.1: User Roles and Their Capabilities

Chapter 5

Object-Oriented Concepts Used

5.1 Classes and Objects

Classes and Objects

A **class** is a blueprint that defines the data and behaviour of a concept. An **object** is a live instance of that class created at runtime.

Every real-world concept in the marketplace is represented by a class. For example, `Product` describes what a product is (its fields), and the methods that operate on it. At runtime, every product in the database corresponds to an individual `Product` object.

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private BigDecimal price;
    private int stock;

    @Enumerated(EnumType.STRING)
    private ListingStatus listingStatus = ListingStatus.
        PENDING_APPROVAL;

    // Getters and setters...

    public void addImage(ProductImage image) {
        images.add(image);
        image.setProduct(this);
    }
```

```
}
```

Listing 5.1: Product Entity Class (simplified)

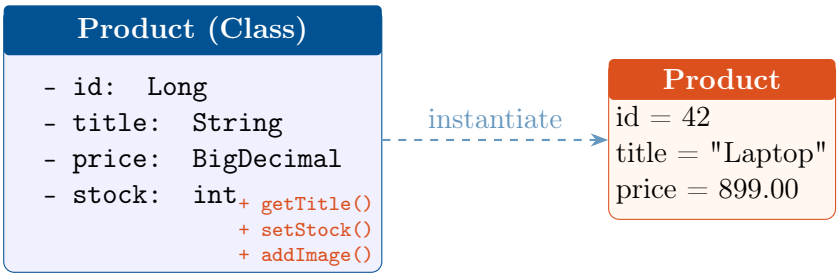

Figure 5.1: Class vs Object: Product

5.2 Encapsulation

Encapsulation

Encapsulation means hiding internal data inside a class and controlling access through public methods (getters and setters). This protects the integrity of an object's state.

Every entity in the domain package follows strict encapsulation. All fields are declared `private`; only the class itself can access them directly. All access from outside goes through well-named getter and setter methods. This means that if validation logic needs to be added to, say, `setStock()`, it can be done in one place without changing any calling code.

```java
@Entity
@Table(name = "users")
public class User implements UserDetails {

    @Column(nullable = false, unique = true)
    private String email;   // private     cannot be accessed directly

    @Column(nullable = false)
    private String password; // never exposed as plain text

    private boolean enabled = true;

    // Controlled access via public methods
    public String getEmail() { return email; }
```

```java

    public void setEmail(String email) {
        this.email = email;
    }

    // Password returned for Spring Security only
    @Override
    public String getPassword() { return password; }
}
```

Listing 5.2: Encapsulation in the User class

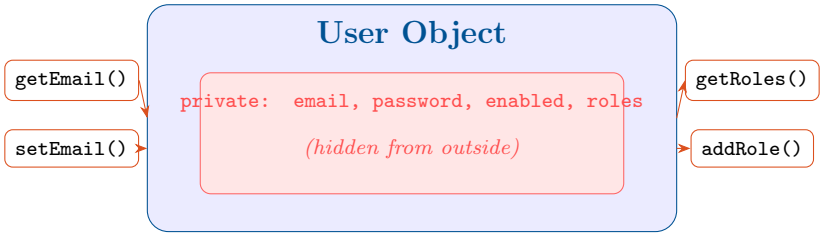

Figure 5.2: Encapsulation: User fields are private, accessed through public methods

5.3 Inheritance

Inheritance

Inheritance allows a class to acquire the fields and methods of another class. In Java this is expressed with the `extends` keyword (for classes) or `implements` (for interfaces).

In this project, the most visible use of inheritance is `BusinessException`, which **extends** Java's built-in `RuntimeException`. This means it automatically inherits all exception-handling behaviour from the standard library, while the project team can add domain-specific constructors and semantics.

```java
public class BusinessException extends RuntimeException {

    public BusinessException(String message) {
        super(message);  // calls parent constructor
    }
}
```

Listing 5.3: BusinessException extends RuntimeException

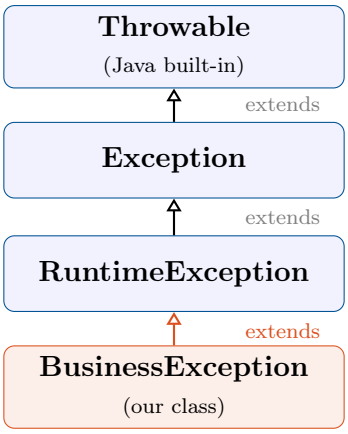

Figure 5.3: Inheritance hierarchy for BusinessException

5.4 Polymorphism

Polymorphism

Polymorphism means that a single reference type can refer to objects of different concrete types. The correct method is chosen at runtime, not compile time.

The clearest example is the `PaymentService` interface and its concrete implementation `MockPaymentService`. Any code that depends on `PaymentService` does not need to know *which* payment provider is in use. In the future, a `StripePaymentService` or `PayPalPaymentService` could be plugged in with zero changes to the calling code.

```
// The interface      a contract
public interface PaymentService {
    void completeMockPayment(Long orderId, User payer);
}

// The concrete class      fulfils the contract
@Service
public class MockPaymentService implements PaymentService {

    @Override
    @Transactional
    public void completeMockPayment(Long orderId, User payer) {
        // ... validates order, deducts stock, marks payment as
           SUCCEEDED
        pay.setStatus(PaymentStatus.SUCCEEDED);
        pay.setReference("MOCK-" + UUID.randomUUID());
        order.setStatus(OrderStatus.PAID);
    }
```

```
}
```

Listing 5.4: PaymentService interface and MockPaymentService implementation

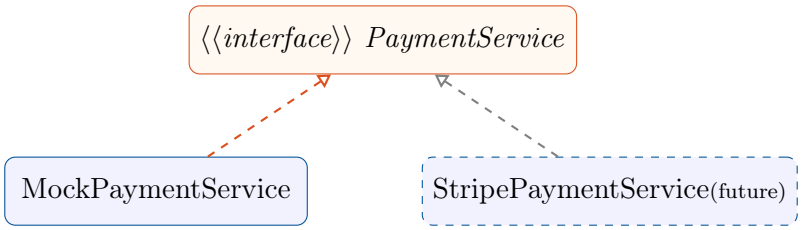

Figure 5.4: Polymorphism via PaymentService interface

5.5 Abstraction

Abstraction

Abstraction means exposing only what a caller needs to know, and hiding the internal details. It reduces complexity by letting you use a system without understanding how it works internally.

The `PaymentService` interface is a perfect example: the checkout controller only calls `completeMockPayment(orderId, payer)` — it does not know about UUID generation, order status transitions, or stock deduction. All that complexity is hidden behind one method signature. Service classes in general act as abstractions over the data layer — controllers never talk directly to repositories.

```
// In ShopperOrderController:
paymentService.completeMockPayment(orderId, currentUser);
// The controller has no idea how payment works internally.
```

Listing 5.5: Abstraction: Controller calls one method, unaware of internals

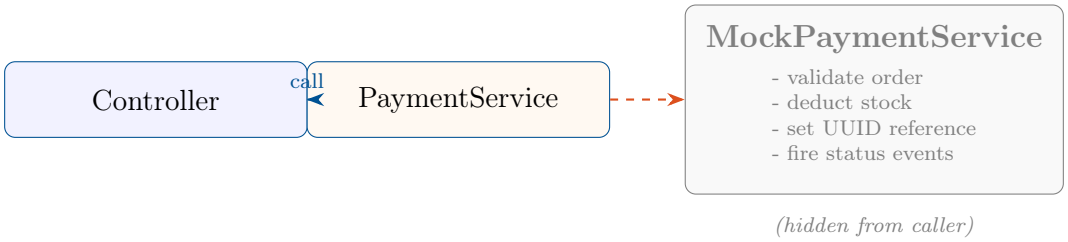

Figure 5.5: Abstraction: controller delegates to a service without knowing its internals

5.6 Interfaces

Interfaces

An interface in Java is a fully abstract type that defines a set of method signatures. Any class that `implements` it must provide concrete implementations.

Two key interfaces are used in this project:

1. **PaymentService** — defines the payment contract. Implemented by `MockPaymentService`. Allows future payment providers to be added easily.
2. **UserDetails** (Spring Security) — defines how Spring Security reads user credentials. The `User` entity implements this interface directly, meaning it can be used by the security framework without any adapter code.

```java
public class User implements UserDetails {

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities()
        {
        return roles.stream()
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r.name()
                ))
            .collect(Collectors.toSet());
    }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked()  { return true; }
}
```

Listing 5.6: User implements UserDetails interface

5.7 Method Overloading and Overriding

Method Overloading vs Overriding

Overloading: Multiple methods in the same class with the same name but different parameters.
Overriding: A subclass or implementing class provides its own implementation of a method declared in its parent or interface.

Method Overriding is used extensively in this project. Every method annotated with `@Override` is an override — for example, all methods implementing `UserDetails` in the `User` class are overrides. The `MockPaymentService.completeMockPayment()` method overrides the abstract contract defined by the `PaymentService` interface.

Method Overloading appears in the `RegistrationService`, which provides two `register` methods: one for shoppers and one for merchants, with different parameter lists.

```java
@Transactional
public void registerShopper(String email, String rawPassword) {
    // registers a basic shopper account
}

@Transactional
public void registerMerchant(String email, String rawPassword,
                             String businessName, String
                                description) {
    // registers a merchant account + creates MerchantProfile
}
```

Listing 5.7: Overloading in RegistrationService

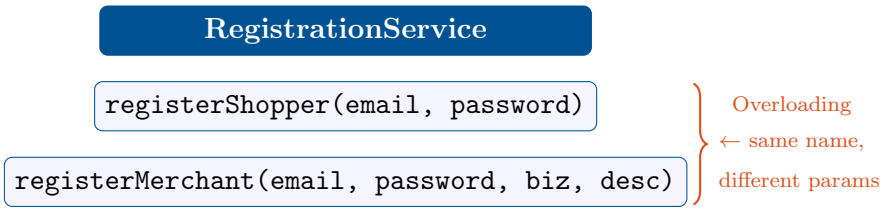

Figure 5.6: Method Overloading in RegistrationService

5.8 Exception Handling

Exception Handling

Exception handling allows a program to detect and respond to errors at runtime without crashing. Java uses `try/catch/throw` and allows defining custom exception types.

The project uses a two-tier exception strategy:

1. **BusinessException** — thrown by service classes whenever a business rule is violated (e.g., empty cart, insufficient stock, unauthorised action).
2. **GlobalExceptionHandler** — a `@ControllerAdvice` class that catches every `BusinessException` thrown anywhere in the application and gracefully redirects the user back with a flash error message, instead of showing an error page.

```java
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public String business(BusinessException ex,
                           HttpServletRequest req,
                           RedirectAttributes ra) {
        ra.addFlashAttribute("error", ex.getMessage());
        String ref = req.getHeader("Referer");
        String base = req.getScheme() + "://" + req.getServerName()
                      + ":" + req.getServerPort();
        if (ref != null && ref.startsWith(base)) {
            return "redirect:" + ref;
        }
        return "redirect:/";
    }
}
```

Listing 5.8: GlobalExceptionHandler catches all BusinessExceptions

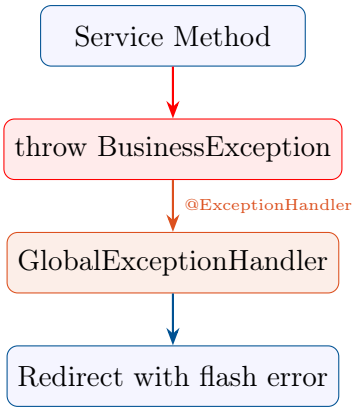

Figure 5.7: Exception Handling Flow

5.9 Static Members

Static Members

A static field or method belongs to the class itself, not to any individual object. It is shared across all instances.

In `ProductService`, the constant `MAX_IMAGES` is declared as `private static final`. This means it is a class-level constant — not stored in each `ProductService` object, just once for the class. Static constants are ideal for configuration values that never change.

```java
@Service
public class ProductService {

    private static final int MAX_IMAGES = 5;  // class-level constant

    private void saveImages(Product product, List<MultipartFile> images)
            throws IOException {
        if (start + images.size() > MAX_IMAGES) {
            throw new BusinessException(
                "At most " + MAX_IMAGES + " images per product.");
        }
        // ...
    }
}
```

Listing 5.9: Static constant in ProductService

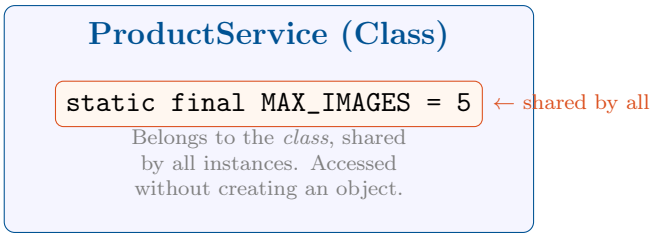

Figure 5.8: Static constant in ProductService

5.10 Collections

Collections

Java Collections (List, Set, Map) are data structures that store multiple objects. OOP-based collections allow storing, iterating, filtering, and transforming groups of domain objects.

Collections are used throughout the domain model:

- `User.roles` is a `Set<Role>` — prevents duplicate roles.
- `Cart.items` is a `List<CartItem>` — ordered list of cart lines.
- `CustomerOrder.lines` is a `List<OrderLine>`.
- `Product.images` is a `List<ProductImage>`, ordered by `sortOrder`.
- The `CartService.addItem()` method uses Java Stream API (filtering, `findFirst()`) to work with collections.

```java
// Find an existing cart item for the same product
Optional<CartItem> line = cart.getItems()
    .stream()
    .filter(ci -> ci.getProduct().getId().equals(productId))
    .findFirst();

if (line.isPresent()) {
    int next = line.get().getQuantity() + quantity;
    line.get().setQuantity(next);
} else {
    CartItem ci = new CartItem();
    ci.setProduct(p);
    ci.setQuantity(quantity);
    cart.addItem(ci);
}
```

Listing 5.10: Collections and Streams in CartService

5.11 Composition and Aggregation

Composition vs Aggregation

Composition: A "whole-part" relationship where the part cannot exist without the whole (strong ownership).
Aggregation: A "has-a" relationship where the part can exist independently.

Composition examples:

- `CustomerOrder` *owns* its `OrderLines` and its `Payment`. If the order is deleted, the lines and payment are deleted with it (`CascadeType.ALL, orphanRemoval = true`).
- `Cart` *owns* its `CartItems`.
- `Product` *owns* its `ProductImages`.

Aggregation examples:

- `OrderLine` *references* a `Product` — if the product is deleted the order line record survives (it stores the product title as a snapshot).
- `Dispute` *references* an order and a user, but does not own them.

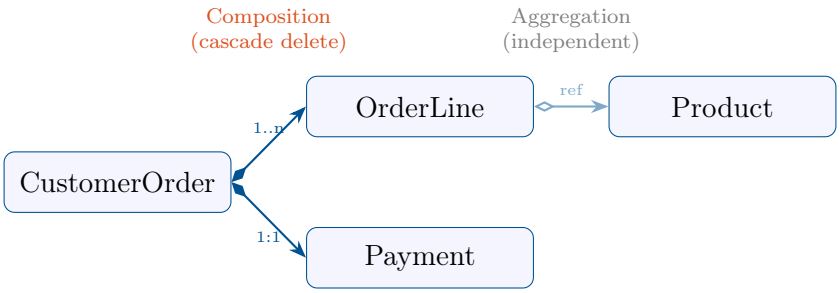

Figure 5.9: Composition and Aggregation in the domain model

Chapter 6

Program Workflow

6.1 Application Startup

When the application starts, Spring Boot performs the following steps automatically:

1. The embedded Tomcat server starts on port 8080.
2. Spring Data JPA connects to the configured database (SQLite or PostgreSQL).
3. Flyway runs any pending migration scripts, creating tables if they do not exist.
4. The `DevDataLoader` bean (in development profile) seeds the database with sample users, a merchant, and products so that the system can be tested immediately.
5. Spring Security initialises its filter chain. Unauthenticated requests to protected routes are redirected to `/login`.

6.2 Authentication

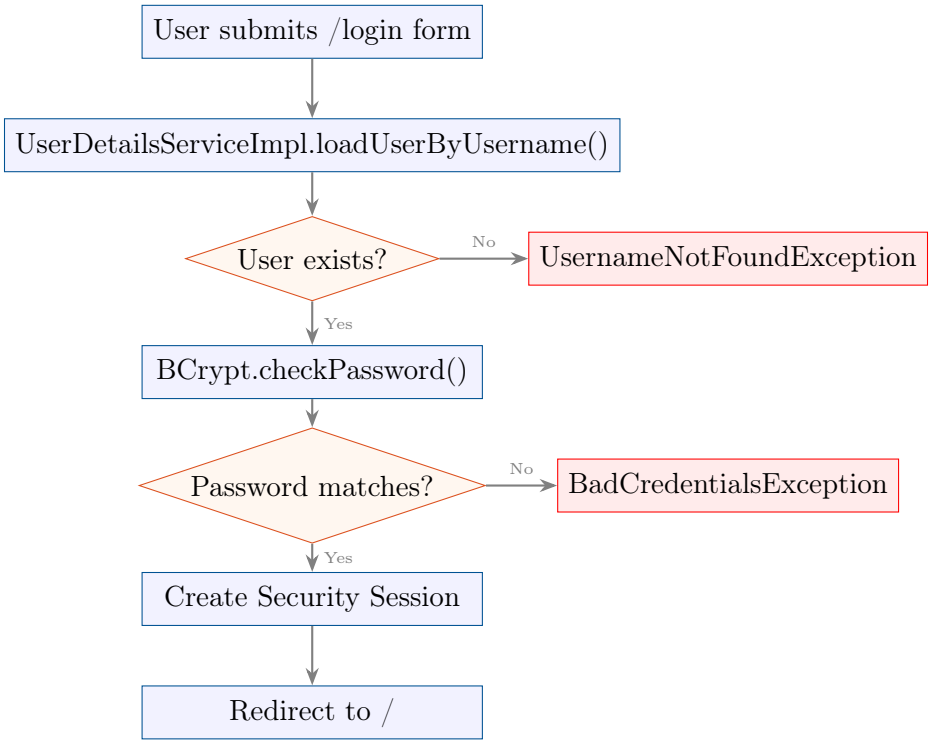

Figure 6.1: Authentication Flow

Authentication is handled entirely by Spring Security. The `UserDetailsServiceImpl` class implements the `UserDetailsService` interface, loading a `User` entity by email. Spring Security then compares the submitted password against the stored BCrypt hash. If the credentials match, a session is created. All subsequent requests are authenticated via this session.

6.3 Main Application Loop

Once authenticated, each user type interacts with a different part of the application:

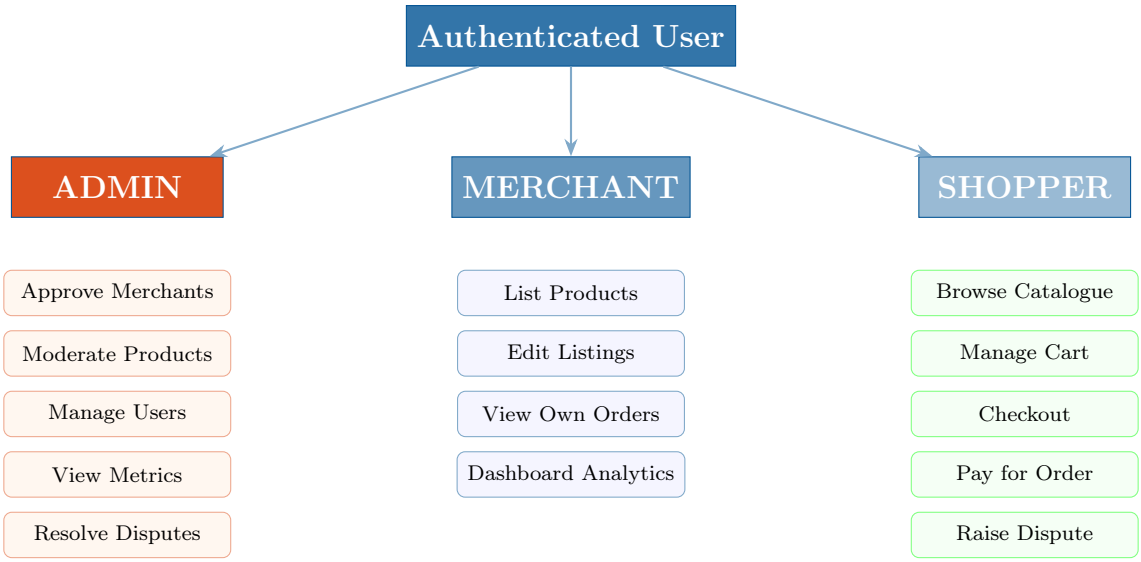

Figure 6.2: Main Loop: User Role to Available Actions

6.4 Operation Flow: Add to Cart

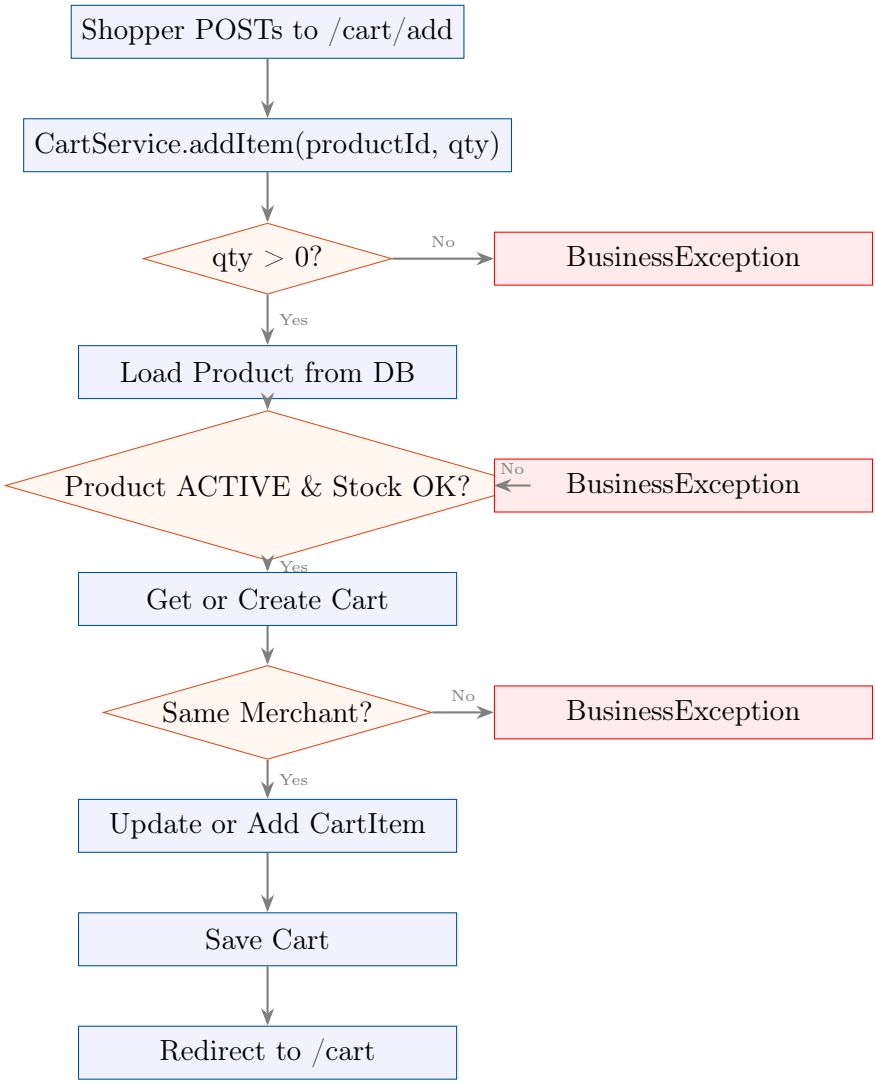

Figure 6.3: Add-to-Cart Operation Flow

Chapter 7

Input and Output Examples

7.1 User Registration

Scenario: A new user registers as a shopper via the web form.

Input (form fields submitted to `POST /register`):

```
email    = "ali@example.com"
password = "Secret123"
role     = SHOPPER
```

Processing:

1. `RegistrationController` receives the form and calls `RegistrationService.registerShopper()`.
2. The service checks that the email is not already taken.
3. The password is hashed with BCrypt.
4. A new `User` entity is created with `roles = {SHOPPER}`.
5. The entity is saved to the `users` table.

Output: User is redirected to `/login` with a success message.

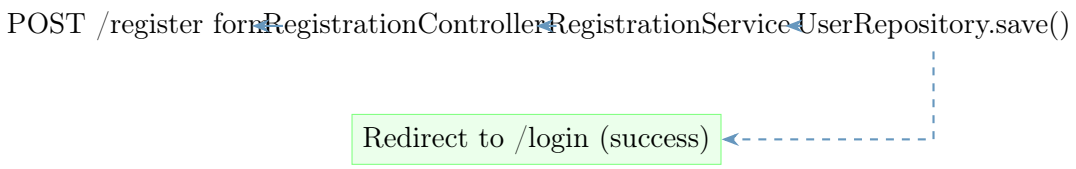

Figure 7.1: Registration Request Flow

7.2 Merchant Adds a Product

Scenario: An approved merchant creates a new product listing.

Input (form submitted to `POST /merchant/products/new`):

```
title       = "Wireless Keyboard"
description = "Compact, Bluetooth 5.0, 3-month battery life."
price       = 45.00
stock       = 100
images      = [keyboard.jpg]
```

Processing:

1. `MerchantController` calls `ProductService.create()`.
2. The service verifies the caller has `ROLE_MERCHANT` and is `APPROVED`.
3. Validates title, price, and stock.
4. Saves the product with status `PENDING_APPROVAL`.
5. Saves the uploaded image to the `uploads/` directory.

Output: Merchant sees their product list. The product appears with status `PENDING_APPROVAL` until an admin activates it.

7.3 Shopper Buys a Product

Scenario: A shopper adds a product to cart and completes checkout and payment.

Step 1 — Add to Cart:

```
POST /cart/add
productId = 7, quantity = 2
```

Step 2 — Checkout:

```
POST /checkout
→ Creates CustomerOrder #19, status = PENDING_PAYMENT
→ Cart cleared
```

Step 3 — Pay:

```
POST /orders/19/pay
→ Stock deducted: -2 units
→ Payment status: SUCCEEDED (MOCK-{uuid})
→ Order status: PAID
```

Figure 7.2: Purchase flow from cart to paid order

7.4 Admin Views Platform Metrics

Scenario: The administrator visits the dashboard.

Input: `GET /admin`

Processing: `AdminMetricsService.snapshot()` computes:

- Total number of registered users.
- Total number of active product listings.
- Total orders placed and total revenue.
- Count of open disputes.
- Count of merchants pending approval.

Output (shown in admin dashboard):

```
Total Users           : 128
Active Products       : 74
Orders (all time)     : 312
Total Revenue         : $14,850.00
Open Disputes         : 3
Merchants Pending     : 2
```

Chapter 8

Conclusion

8.1 What Was Built

This project resulted in a fully functional multi-role online marketplace system built in Java 17 using the Spring Boot framework. The system supports three user roles — shopper, merchant, and admin — each with a dedicated set of features. Shoppers can browse a product catalogue, manage a shopping cart, place orders, and pay using a mock payment service. Merchants can register, await admin approval, and then manage their product listings including uploading images. Administrators can approve merchants, moderate product listings, manage user accounts, monitor platform metrics, and resolve disputes.

8.2 OOP Usage

Every major OOP principle studied in this course was applied in a practical, real-world context:

- **Encapsulation** — all domain entities use private fields with accessor methods.
- **Inheritance** — `BusinessException` extends `RuntimeException`.
- **Polymorphism** — `MockPaymentService` implements `PaymentService`.
- **Abstraction** — service classes hide complexity from controllers.
- **Interfaces** — `PaymentService` and `UserDetails` are both used.
- **Exception Handling** — centralised via `GlobalExceptionHandler`.
- **Static Members** — `MAX_IMAGES` constant.
- **Collections** — `List` and `Set` throughout.
- **Composition/Aggregation** — clearly expressed via JPA cascade rules.

8.3 System Strengths

- **Clean layer separation** — no business logic leaks into controllers or templates.
- **Extensible payment layer** — swapping in a real payment provider requires only a new `PaymentService` implementation.
- **Robust error handling** — all exceptions are caught centrally and surfaced gracefully.
- **Role-based security** — enforced at the URL level and method level by Spring Security.
- **Database portability** — works with both SQLite and PostgreSQL via profile switching.
- **Test coverage** — key business logic is tested at the service and controller level.

Chapter 9

Future Improvements

9.1 Real Payment Gateway Integration

Current Limitation: The current payment system uses a mock implementation (`MockPaymentService` that simulates a successful payment without any real financial transaction. While this is perfectly adequate for demonstrating OOP concepts, it means the system cannot be deployed commercially as-is.

Proposed Solution: The `PaymentService` interface already makes this upgrade straightforward. A new class, `StripePaymentService`, can be created that implements the same interface but uses the Stripe Java SDK to initiate a real charge. The key architectural change would be adding a redirect flow: when the user clicks "Pay", they are sent to a Stripe-hosted checkout page, and Stripe sends a webhook back to the server once the payment succeeds or fails. The server listens to these webhooks and transitions the order status accordingly. This introduces the concept of asynchronous operations — the payment does not complete synchronously within the HTTP request.

Expected Benefit: The marketplace becomes capable of processing real transactions. Merchants would receive payments, and the platform could take a small commission fee. Trust in the platform increases significantly because users interact with a known and trusted payment provider.

Code Snippet:

```java
@Service
@Profile("stripe")
public class StripePaymentService implements PaymentService {

    @Value("${stripe.secret-key}")
    private String stripeSecretKey;

    @Override
    public void completeMockPayment(Long orderId, User payer) {
        // Create Stripe PaymentIntent
```

```
        Stripe.apiKey = stripeSecretKey;
        PaymentIntentCreateParams params =
            PaymentIntentCreateParams.builder()
                .setAmount(amountInCents)
                .setCurrency("usd")
                .build();
        PaymentIntent intent = PaymentIntent.create(params);
        // store intent.id as the payment reference
    }
}
```

Listing 9.1: Skeleton of StripePaymentService

9.2 Product Search and Filtering

Current Limitation: The current catalogue controller (`CatalogController`) displays a list of active products but offers only basic functionality. The project already includes a `ProductSpecifications` class suggesting that JPA Specifications were planned, but a full search and filtering UI has not been implemented.

Proposed Solution: A comprehensive search module should be added. This would include a search bar at the top of the catalogue page that queries products by keyword (matching title or description). Faceted filtering would allow shoppers to narrow results by price range, merchant, and availability. Sorting options (by price ascending/descending, newest first) should also be provided. On the backend, the `ProductSpecifications` class can be expanded to compose dynamic JPA predicates based on the search parameters submitted by the user. For larger datasets, Elasticsearch integration could be considered to provide full-text search with relevance ranking.

Expected Benefit: Better discoverability of products leads directly to higher conversion rates and a better user experience. Shoppers can find what they are looking for quickly instead of scrolling through an undifferentiated list.

9.3 Product Review and Rating System

Current Limitation: There is currently no way for shoppers to rate or review products after purchasing them. This is a significant gap because reviews are one of the most important trust signals on an e-commerce platform.

Proposed Solution: A new `ProductReview` entity should be created, with fields for the rating (integer 1–5), a text comment, the reviewer (a `User` who has a `PAID` order containing

the product), and the timestamp. The relationship between `Product` and `ProductReview` is one-to-many. A constraint should be enforced at the service level: a shopper can only review a product they have actually purchased, and only once per product. The product detail page should display an average rating and a list of recent reviews.

Expected Benefit: Reviews build trust for new shoppers who have not purchased from a merchant before. Merchants benefit from positive feedback, and the admin team can use rating data to identify underperforming merchants.

9.4 Merchant Analytics Dashboard

Current Limitation: The merchant dashboard exists (`MerchantAnalyticsService` is present) but the analytics exposed are limited. A merchant cannot easily see which products are selling best, revenue trends over time, or how their conversion rates look.

Proposed Solution: The analytics service should be significantly expanded. It should compute: total revenue per merchant per month, top five best-selling products, average order value, and total units sold per product. These metrics should be displayed on a visual dashboard using a charting library such as Chart.js (included via a CDN in Thymeleaf templates). The underlying data should be computed using JPQL aggregate queries (GROUP BY, SUM, COUNT) in the repository layer, which gives the application high performance without pulling large amounts of data into memory.

Expected Benefit: Merchants can make informed decisions about pricing, restocking, and which products to promote. Better merchant experience makes the platform more attractive for new sellers.

9.5 Email Notifications

Current Limitation: The system performs all actions silently — when an order is placed, when a merchant is approved, or when a dispute is resolved, no notification is sent to the relevant user. Users must log in to discover what has happened.

Proposed Solution: Spring Boot provides `spring-boot-starter-mail` for sending emails. An `EmailService` interface should be created (following the same pattern as `PaymentService`), with methods like `sendOrderConfirmation()`, `sendMerchantApproved()`, and `sendDisputeResolved()`. Implementations can use JavaMailSender or integrate with a transactional email service like SendGrid or Mailgun. Thymeleaf templates can be used to generate rich HTML email bodies. Key events that should trigger emails include: order placed (to shopper), order paid (to merchant), merchant application approved or declined, dispute raised, and dispute resolved.

Expected Benefit: Users stay informed without having to manually check the platform. This is a standard expectation for any e-commerce system. Email notifications also give the platform a professional, polished feel that increases user trust.

9.6 REST API and Mobile Support

Current Limitation: The current system is a server-rendered web application. All interaction is through Thymeleaf templates, which are designed for browsers. There is no API that a mobile app or a third-party integration could use.

Proposed Solution: A parallel REST API layer should be introduced. Spring Boot makes this easy: new `@RestController` classes can be added alongside the existing `@Controller` classes. The REST controllers would accept JSON requests and return JSON responses, reusing the same service layer that the web controllers already use. Authentication for the API should use JWTs (JSON Web Tokens) rather than session cookies, since mobile apps and API clients do not maintain sessions. A library like Spring Security with JWT (JJWT library) can generate and validate tokens on each request. API endpoints should be versioned (`/api/v1/...`) so that breaking changes can be introduced without affecting existing clients.

Expected Benefit: A REST API opens the platform to mobile applications (iOS, Android), allows third-party developers to build integrations, and is a prerequisite for any modern B2B use case. This is a major step towards making the marketplace a platform product rather than just a website.

Chapter 10

References

The following references informed the design decisions, coding practices, and theoretical foundations used in this project. Each entry includes the source type, author, title, year, and a note explaining how it relates to the project.

[1] Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley. URL: https://www.informit.com/store/effective-java-9780134685991
This book guided the design of our entity classes, especially advice on minimising the public API of classes (Encapsulation), using interfaces for flexibility, and preferring composition over inheritance. The design of `BusinessException` *was inspired by Item 60 (favour standard exceptions) and Item 70 (use checked exceptions sparingly).*

[2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.
Known as the "Gang of Four" book. The service interface pattern used for `PaymentService`*/*`MockPayment` *follows the Strategy design pattern described here. The* `GlobalExceptionHandler` *is inspired by the Template Method and Chain of Responsibility patterns.*

[3] Horstmann, C. S. (2022). *Core Java, Volume I — Fundamentals* (12th ed.). Oracle Press. URL: https://horstmann.com/corejava/
Used as the primary reference for Java language fundamentals including collections, generics, and the Stream API. The streaming operations in `CartService` *and* `User.getAuthorities()` *were written based on patterns from this text.*

[4] Spring Framework Team. (2024). *Spring Framework Documentation 6.1.* VMware. URL: https://docs.spring.io/spring-framework/reference/
The definitive reference for all Spring features used in the project including dependency injection, Spring MVC, Thymeleaf integration, and transaction management.

[5] Spring Security Team. (2024). *Spring Security Reference.* VMware. URL: https://docs.spring.io/spring-security/reference/
Used to implement the `SecurityConfig` *class, configure role-based access control, BCrypt password encoding, and form-login setup. The approach of implementing* `UserDetails` *on the* `User` *entity was recommended in this documentation.*

[6] Spring Data Team. (2024). *Spring Data JPA Reference.* VMware. URL: https://docs.spring.io/spring-data/jpa/reference/
The Spring Data JPA repository interfaces (`JpaRepository`, `JpaSpecificationExecutor`) used in this project are documented here. The `ProductSpecifications` class was written following the Specifications pattern described in this reference.

[7] Flyway Team. (2024). *Flyway Documentation.* Redgate. URL: https://documentation.red-gate.com/flyway
Flyway was used for database schema migration management. The SQL scripts in `src/main/resources/` follow the versioning convention documented here.

[8] Fowler, M. (2002). *Patterns of Enterprise Application Architecture.* Addison-Wesley. URL: https://martinfowler.com/books/eaa.html
The four-layer architecture (Presentation, Service, Domain, Data) follows the Layered Architecture pattern described in this book. The concept of a Service Layer handling all business logic is a direct application of Fowler's recommendations.

[9] Oracle. (2023). *Java SE 17 API Documentation.* Oracle Corporation. URL: https://docs.oracle.com/en/java/javase/17/docs/api/
The Java SE 17 API documentation was consulted throughout the project for standard library classes including `java.util.Optional`, `java.util.Set`, `java.math.BigDecimal`, and `java.time.Instant`.

[10] Jakarta EE Team. (2023). *Jakarta Persistence 3.1 Specification.* URL: https://jakarta.ee/specifications/persistence/3.1/
The JPA annotations used throughout the domain classes (`@Entity`, `@ManyToOne`, `@OneToMany`, `CascadeType.ALL`, `orphanRemoval`) are defined in this specification. The decision to use `CascadeType.ALL` with `orphanRemoval` for composition relationships was based on the lifecycle management semantics described here.

[11] Baeldung. (2024). *Spring Boot Tutorials and Articles.* Baeldung. URL: https://www.baeldung.com/spring-boot
Baeldung tutorials were consulted for practical implementation patterns including the use of `@ControllerAdvice` for exception handling, configuring multiple datasource profiles, and writing Spring Security test cases with `@WithMockUser`.

[12] Walls, C. (2022). *Spring Boot in Action* (2nd ed.). Manning Publications. URL: https://www.manning.com/books/spring-boot-in-action
This book provided practical guidance on auto-configuration, application profiles, and the overall structure of a Spring Boot project. The `application.yml` configuration approach and the use of `@Value` for injecting configuration properties were taken from patterns in this text.

[13] Deitel, P., & Deitel, H. (2020). *Java: How to Program, Early Objects* (11th ed.).

Pearson.
A foundational textbook covering all core OOP concepts including encapsulation, inheritance, polymorphism, and abstract classes. The theoretical foundations for the OOP concepts chapter of this report are based on definitions and examples from this text.

[14] SQLite Consortium. (2024). *SQLite Documentation.* SQLite.org. URL: https://www.sqlite.org/docs.html
SQLite was used as the development database. The dialect configuration for Hibernate's community SQLite support was referenced from this documentation.

[15] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation.* URL: https://www.postgresql.org/docs/16/
PostgreSQL is the production database profile for this project. The SQL scripts in the `postgresql/` *migration folder use PostgreSQL-specific syntax documented here.*